

Hardware-Software Interface

F28HS

Coursework 2

Heriot-Watt University

Edinburgh Campus

Sailesh Aduri

Ashaz Asaf

Problem Specification

This project implements a MasterMind game on a Raspberry Pi using two LEDs (red and green), a button, and an LCD. The game logic is written in C, while inline ARM Assembler handles low-level GPIO operations—reading button presses, controlling LEDs, and driving the LCD. The matching function is also written on a separate file purely in ARM Assembler. The Assembler functions are linked to the C program via a Makefile, ensuring efficient hardware interaction. Players input guesses via button presses, with LEDs providing feedback (exact/approximate matches). The LCD displays results. Debug mode reveals the secret sequence at the start of the game including user guesses and answers on the terminal. This mode works well with the secret mode with flag '-s' that allows us to define the secret sequence. Unit tests on the other hand verify the matching logic and use '-u'. The system demonstrates embedded programming principles, combining C for game flow and Assembler for direct hardware control.

Hardware Specification

Hardware Component	Pin (BCM)
Green LED	26
Red LED	5
Button	19
LCD 4	25
LCD 6	24
LCD 11	23
LCD 12	10
LCD 13	27
LCD 14	22

The wiring is close to the one in specification, made a few adjustments where LCD was directly hooked on to the breadboard, less wires needed for that.

Short discussion of code structure

The main files include:

- master-mind.c
- lcdBinary.c
- mm-matches.s

mastermind.c:

In the file master-mind.c, all functions are defined before the main function.

- `initSeq`: called when the game is run without a given sequence. In the main flow right before the game starts, it is called to set a random secret sequence.
- `showSeq`: called only in debug mode so that when we are testing the program it allows the secret to be displayed
- `countMatches`: takes two sequences and is expected to return a pointer to two integer values, one being the exact matches and the other being the approximate matches. Makes use of a function "extern" that is the external mm-matches.s file that uses full ARM Assembly to perform the algorithm
- `showMatches`: After a complete guess, shows the outcome of `countMatches` in unit test mode
- `readSeq`: Is used to read a sequence of values entered as one integer and separates each digit of the integer as an array. Used when unit testing and/or secret (-s) mode
- `Timer_handler` and `initTimer`: allows to set a 3 second timer used in the main function for when the user presses the button. The `initTimer` is used to reset the timer in the case that the button is pressed for more user inputs. The `time_handler` function sets `timed_out` to true indicating that the time expired and it's time to process the data
- `delay`: Used in the main function and `blinkN` to allow the LED lights to stay noticeably on for every blink indicating to users they have been turned on
- `blinkN`: Calls `writeLED` in a loop to turn the lights on and off with the use of `delay`

lcdBinary.c:

Its interfaces for all its functions are found in master-mind.c so that master-mind.c can access the following functions (for example in `blinkN`). These functions written in inline ARM Assembler allow GPIO control and access to the hardware components connected to the RaspberryPi.

- `digitalWrite` and `writeLED`: Writes a high or low value to turn on or off and called for the LCD and LED
- `pinMode`: called to set the LEDs to output and the button to input
- `readButton`: Reads the value of the button where it returns true if it is pressed and false if not to indicate in the main function when the game should start taking input.
- `waitForButton`: will use `readbutton` to not perform further processes until `readButton` returns true.

mm-matches.s:

Extern is used in master-mind.c to link with the external file mm-matches.s which is written in ARM Assembly required for counting the exact and approximate matches and returning them in a pointer to a pair of values

Performance-relevant design decisions covering inline assembly and assembly.

The lcdBinary.c file uses inline ARM and is responsible for manipulating GPIO pins through low-level assembly code.

Functions include:

1. digitalWrite(gpio, pin, value) : void
 - a. Performance -> manipulates just the gpio address, uses 4 registers, 2 blocks.
2. pinMode(gpio, pin, mode) : void
 - a. How it works -> Subtracts pin by a constant and keeps track of remainder and quotient, quotient is equal to register offset, remainder is equal to calculate bit offset.
 - b. Performance -> uses 9 registers as a balance between keeping track of quotient and remainder, storing shift position, multiplying by 3.
3. writeLED(gpio, led, value) : void
 - a. Performance -> same as digitalWrite(), essentially wraps around this function.
4. readButton(gpio, button) : int
 - a. Performance -> It's a simple implementation that stores addresses in registers which only gets used 3 times.

The mm-matches file uses a pure assembly file responsible for returning exact and approximate values.

Performance: all 13 registers are used; it's a balance between a lot of variables to keep track of and supply of registers. The code by itself runs in $O(n^2)$.

Performance-relevant design decisions covering master-mind.c

To minimize overhead, the signal handler installation for SIGALRM which was initially set in initTimer was moved to main to prevent repeated unneeded reinstallation of the signal handler whenever the timer rest, making the program more efficient.

We have utilised memcpy() to create backups of the sequences in countMatches before calling mm-matches.s, ensuring the original data is preserved. This allows the program to reliably reprocess the same sequences without corruption or unexpected modifications.

List of functions directly accessing the hardware

Directly accessing hardware in ARM, digitalWrite(), pinMode(), writeLED(), readButton().

Directly accessing hardware through C to ARM : blinkN(), lcdPut(), waitForButton(), main(), lcdPutChar(), lcdPutCommand(), sendDataCmd().

Name and interface discussion

The matching function is also included in C within the first lines of the master-mind.c file.

Exact working methodology is explained in the code, it is too sophisticated to explain without references in the word document.

Overview: The code uses 2 inputs, stored in r0 and r1. Initializes 2 arrays which record where exact and approx digits were guessed. R0 stores secret sequence R1 stores guess sequence. Return type is a pointer to an array 4 bits apart, storing 2 int values, exact and approx..

In each of mainARM and matches, registers are pushed to the stack to preserve their value, except for r4 and r5 which store usedExact and usedApprox arrays, which will be later reset at the end of matches.

Blocks of code:

1. 3 loops -> 1st loop iterates over to calculate exact count, 2nd loop has an outer and inner loop which calculate approx count.
2. 2 functions -> mainARM (purpose is to initialize the array), matches (purpose is to run these loops and call mainARM).

Run-down of subroutines inside matches:

1. exact_loop: a for loop, utilizes exact_skip to jump straight to next iteration after incrementing count variable.
 - a. R11 -> secret[i], R12 -> guess[i]
 - b. If R11 == R12 then increment 'exact' count.
2. Outer_loop: a for loop, which skips over values which are exact in secret sequence from guess sequence. Skips iteration if current value in guess was already found out to be an exact value.
3. Inner_loop: a for loop, which increments approx. count based on 3 conditions
 - a. usedApprox[j] (j loop variable for inner loop) = 0
 - b. usedExact[j] = 0
 - c. secret[i] (i loop variable for outer loop) == guess[j]
 - d. There are skip sub-routines for each loop where the loop counter is incremented without going over the if statements.
4. Load 'result' from data which is a word containing 2 placeholders for 2 values.
 - a. R2 stores exact
 - b. R3 stores approx.
 - c. Load both of them into the result then move the result to r0.
 - d. Pop all the registers back to restore original values then return.

Example set of output from complete game.

```
ama2026@raspberrypi:~/Desktop/f28hs-2024-25-cwk2-sys-final $ sudo ./cw2 -d -s 123
Raspberry Pi LCD driver, for a 16x2 display (4-bit wiring)
Printing welcome message on the LCD display ...
Secret: 1 2 3
Guess1: 1 3 3
Answ1:          2 0
Guess2: 1 2 3
Answ2:          3 0
ama2026@raspberrypi:~/Desktop/f28hs-2024-25-cwk2-sys-final $ sudo ./cw2 -u 122 123
2 exact
0 approximate
ama2026@raspberrypi:~/Desktop/f28hs-2024-25-cwk2-sys-final $ sudo ./cw2 -u 123 123
3 exact
0 approximate
ama2026@raspberrypi:~/Desktop/f28hs-2024-25-cwk2-sys-final $
```

Figure 1

At first the unit test resulted in a few false results as we initially programmed it in C. Once we figured out the issue, which was that we needed to use two extra arrays to mark used values of the sequences, we resolved the problem resulting in accurate results. Finally, we translated it into the ARM Assembly. Tested to ensure output in cases for all exact, all approx, one exact and two approx, two exact and one approx and finally zero exacts and zero approx all resulting in accurate answers.

Summary

We have implemented codes which cover different ways to use assembly in c, inline and pure assembly. We used assembly to directly access and manipulate GPIO addresses (initially done in C). Implemented debug mode for testing game functionality without hardware and other modes of command-line usage.

However, we did not use LCD to its full potential (could've added button counts on display). Did not explore other ways to implement matching function, could've been more efficient if more time was spent by developers (us). 'testm.c' file was not used at all so not all was done but we understand the limitations of software testing by not using hardware just CMD inputs to reduce time for hardware setup.

We have learnt how we integrate two different programming languages together and form one complete program that runs a game. Furthermore, different ways we can test out the functionality of our program using different modes (debug, unit testing, secret modes).